Lenguaje Python

Cursada 2025

Clase 2: listas, funciones y módulos

En el video de cadenas planteamos el siguiente desafío:

> Escribir un programa que ingrese 4 palabras desde el teclado e imprima aquellas que contienen la letra r.

Una posible solución:

In [ ]:
```
for i in range(4):
    word = input("Ingresá una palabra: ")
    if "r" in word:
        print(f"{word} tiene una letra r")
```

- ¿Se acuerdan qué representa **f"{word} tiene una letra r"**?

DESAFÍO 1

> Vamos a modificar el código anterior para que se imprima la cadena "TIENE R" si la palabra contiene la letra r y sino, imprima "NO TIENE R".

La solución casi immediata:

In [ ]:
```
for i in range(4):
    word = input("Ingresá una palabra: ")
    if "r" in word:
        print("TIENE R")
    else:
        print("NO TIENE R")
```

Hay otra forma de escribir estas expresiones condicionales.

Expresión condicional

- La forma general es:

```
A if C else B
```

- Devuelve A si se cumple la condición C, sino devuelve B.

In [ ]:
```
number1 = int(input("Ingresá un número: "))
number2 = int(input("Ingresá un número: "))
```

```
number_max = number1 if number1 > number2 else number2
number_max
```

Escribimos otra solución al desafío

In [ ]:
```
for i in range(4):
    word = input("Ingresá una palabra: ")
    print("TIENE R" if "r" in word else "NO TIENE R")
```

Evaluación del condicional

IMPORTANTE: Python utiliza la **evaluación con circuito corto** para evaluar las condiciones

In [ ]:
```
number1 = 1
number2 = 0

if True or number1/number2:
    print("No hay error!!!!???")
```

DESAFÍO 2

> Ingresar palabras desde el teclado hasta ingresar la palabra FIN. Imprimir aquellas que empiecen y terminen con la misma letra.

- ¿Qué estructura de control deberíamos utilizar para realizar esta iteración? ¿Podemos utilizar la sentencia **for**?

Iteración condicional

- Python tiene una sentencia **while**. ¿Se acuerdan de este ejemplo?

In [ ]:
```
## Adivina adivinador....
import random

random_number = random.randrange(5)
i_won = False
print("Tenés 2 intentos para adivinar un entre 0 y 4")
tries = 1

while tries < 3 and not i_won:
    entered_number = int(input("Ingresa tu número: "))
    if entered_number == random_number:
        print("Ganaste!")
        i_won = True
    else:
        print("Mmmm ... No.. ese número no es... Seguí intentando.")
        tries += 1
if not i_won:
    print("Perdiste :(")
    print(f"El número era: {random_number}")
```

La sentencia while

```
while condition:
    sentence
    sentence
```

Nuestro desafío:

> Ingresar palabras desde el teclado hasta ingresar la palabra FIN. Imprimir aquellas que empiecen y terminen con la misma letra.

In [ ]:
```
# Solución al desafío
```

DESAFÍO 3

> Vamos a trabajar con las **películas argentinas candidatas al premio Oscar a mejor película extranjera**.

Queremos saber:

- ¿Cuál fue la duración promedio, en minutos, de todas las películas?
- ¿Cuántas películas duran más que el promedio, en minutos?

Datos según **ChatGPT**:

```
Año de la ceremonia, Película, Resultado, Duración (minutos)
1975, La tregua, Nominada, 108
1985, Camila, Nominada, 105
1986, La historia oficial, Ganadora, 112
1999, Tango, no me dejes nunca, Nominada, 115
2002, El hijo de la novia, Nominada, 123
2010, El secreto de sus ojos, Ganadora, 129
2015, Relatos salvajes, Nominada, 122
2023, Argentina, 1985, Nominada, 140
```

¿Cómo sería un pseudocódigo de esto?

```
Ingresar la duración de las películas.
Calcular el promedio.
Calcular cuántas películas duran más que el promedio.
```

¿Cómo hacemos el **tercer** proceso? ¿Tenemos que ingresar las duraciones nuevamente?

Obviamente no. **Necesitamos tipos de datos que nos permitan guardar muchos valores**.

Listas

> Una **lista** es una colección ordenada de elementos.

In [ ]:
```
movies_duration = [108, 105, 112, 115, 123, 129, 122, 140]

type(movies_duration)
```

> Las listas son estructuras heterogéneas, es decir que **pueden contener cualquier tipo de datos**, inclusive otras listas.

In [ ]:
```
my_list = [1, "dos", [3, "cuatro"], True]
my_list
```

¿Se acuerdan de este método de **str**?

In [ ]:
```
my_list = "Somos campeones del  mundo!!!".split()
my_list
```

El **método split**, retorna una **lista** con los elementos de la cadena de caracteres.

Operaciones básicas

¿Cuántos elementos tiene la lista?

In [ ]:
```
len(my_list)
```

¿Cómo agrego un elemento?

Una forma:

In [ ]:
```
my_list.append("NUEVO")
my_list
```

¿Cómo elimino un elemento?

Una forma:

In [ ]:
```
my_list.pop(1)
my_list
```

Accediendo a los elementos de una lista

- Se accede a través de **un índice** que indica la posición del elemento dentro de la lista **encerrado entre corchetes** [].
- **IMPORTANTE:** al igual que las cadenas los índices comienzan en 0.

In [ ]:
```
my_list = [ 17, "hola", [1, "dos"], 5.5, True]
print(my_list[0])
print(my_list[2][1] )
print(my_list[-3])
```

Las listas son datos MUTABLES. ¿Qué quiere decir esto?

In [ ]:
```
my_list
```

In [ ]:
```
my_list[0] = ["El Dibu", 23]
my_list
```

Recorriendo una lista

- ¿Qué estructura les parece que podríamos usar?

In [ ]:
```
for elem in my_list:
    print(elem)
```

Podríamos pensar en otra forma de recorrer la lista

In [ ]:
```
list_len = len(my_list)
for i in range(list_len):
    print(my_list[i])
```

¿Por qué no **range(list_len - 1)**?

Asignación de listas

Observemos el siguiente ejemplo:

In [ ]:
```
rock = ["Riff", "La Renga", "La Torre"]
blues = ["La Mississippi", "Memphis"]
music = rock
rock[0] = "Rata Blanca"
print(music)
```

- Recordemos que las variables son **referencias a objetos**.

¿Cuál es el problema?

- Cada objeto tiene un identificación.

In [ ]:
```
#Recordemos que music = rock
print(id(music))
print(id(rock))
print(id(blues))
```

In [ ]:
```
rock.append("Rata Blanca")
music
```

Observemos este código

In [ ]:
```
more_music = rock[:]
print(id(rock))
print(id(more_music))
print(id(music))
```

In [ ]:
```
more_music.append("Hermetica")
rock
```

- **music = rock**: music y rock referencian al mismo objeto (misma zona de memoria).
- **more_music = rock[:]**: more_music y rock referencian a dos objetos distintos (dos zonas de memoria distintas con el mismo contenido).

Observemos este código:

In [ ]:
```
# Otra forma
other_rock = rock.copy()
id(other_rock)
```

- ¿Qué podemos decir del método **copy**?

Operaciones con listas

- Las listas se pueden concatenar (+) y repetir (*)

In [ ]:
```
rock = ["Riff", "La Renga", "La Torre"]
blues =["La Mississippi", "Memphis"]

music = rock + blues
more_rock = rock * 3
```

In [ ]:
```
more_rock
```

Algunas cosas para prestar atención

Analicemos el siguiente código:

In [ ]:
```
my_list = [[1,2]] * 3
my_list
```

In [ ]:
```
my_list[0][1] = 'cambio'
my_list
```

- ¿Qué es lo que sucedió?
  - El operador * repite la **misma lista**, no genera una copia distinta; es el mismo objeto referenciado 3 veces.

Probemos ahora:

In [ ]:
```
my_list = [[1,2], [1, 2], [1, 2]]
my_list[0][1] = 'cambio'
my_list
```

CONSIGNA: probar en casa más operaciones sobre listas

- Algunos métodos o funciones aplicables a listas: **extend()**, **index()**, **remove()**, **count()**.
- +Info en la documentación oficial.
- ¿Qué pueden decir de las siguientes asignaciones?

In [ ]:
```
strange_list = [1, "dos", [3, "cuatro"]]
list1 = strange_list.copy()
```

```
list2 = strange_list[:]
```

¿Qué pasa con la lista original si ejecuto las siguientes instrucciones?

In [ ]:
```
list1[1] = 0
list1[2][0] = 0
```

In [ ]:
```
list2[1] = 10
list2[2][1] = 10
```

Algo muy interesante: comprensión de listas (**list comprehension**)

Observemos el siguiente código: ¿qué elementos contiene la lista **codes**?

In [ ]:
```
import string

letters = string.ascii_uppercase
codes = []
for letter in letters:
    codes.append(ord(letter))
print(codes)
```

Ahora usamos list comprehension

In [ ]:
```
letters = string.ascii_uppercase
codes = [ord(n) for n in letters]
print(codes)
```

¿Y en este otro caso? ¿Cuáles son los elementos de **squares**?

In [ ]:
```
squares = [num**2 for num in range(10) if num % 2 == 0]
squares
```

DESAFÍO 4

> Dada una lista de palabras, **usando list comprehension** generar otra lista con aquellos verbos en infinitivo.

IMPORTANTE: solo vamos a comprobar que terminen en "ar", "er" o "ir".

In [ ]:
```
#Una posible solución
words = ["c", "ir", "sol", "cantar","correr"]
verbs = [pal for pal in words if pal.endswith(("ar", "er", "ir"))]
verbs
```

Retomamos el DESAFIO 3 de las películas

Nuestro desafío era:

> Queremos procesar las duraciones de las películas argentinas candidatas al premio Oscar a película extrajera.

Queremos saber:

- ¿Cuál fue la duración promedio, en minutos?
- ¿Cuántas películas duran más que el promedio, en minutos?

La solución planteada era:

```
Ingresar la duración de las películas.
Calcular el promedio.
Calcular cuántas películas duran más que el promedio.
```

Primer paso: generamos la lista con las duraciones

Si no sabemos cuántas películas vamos a procesar, vamos a suponer que ingresamos una duración igual a 0 para finalizar el ingreso de datos.

¿Cómo hacemos la iteración?

In [ ]:
```
#Una posible solución
minutes = int(input("Ingresá la duración de una película (0 para finalizar): "))
movies_duration  = []
while minutes != 0:
    movies_duration.append(minutes)
    minutes = int(input("Ingresá la duración de una película (0 para finalizar): "))

movies_duration
```

Debemos partir de una lista vacía: []

Segundo paso: calculamos el promedio

In [ ]:
```
# Calculamos el promedio
total = 0
for movie in movies_duration:
    total +=movie

average = total / len (movies_duration)
average
```

In [ ]:
```
#Otra solución

average = sum(movies_duration) / len(movies_duration)
average
```

¡IMPORTANTE! Acá no estamos chequeando que la lista esté vacía!!!

Tercer paso: calculamos cuántas duran más que el promedio

In [ ]:
```
#movies_duration = [108, 105, 112, 115, 123, 129, 122, 140]
#average = sum(movies_duration) / len(movies_duration)

max_average = [n for n in movies_duration if n > average]
print(f"{average = } - {max_average = }")
```

In [ ]:
```
print(f"{len(max_average)} películas duran más que el promedio")
```

Funciones en Python: una forma de definir procesos o subprogramas

Veamos un pseudocódigo de la solución del **DESAFÍO 3**:

```
Ingresar la duración de las películas.
Calcular el promedio.
Calcular cuántas películas duran más que el promedio.
```

Podríamos pensar en dividir en tres procesos:

- En Python, usamos **funciones** para definir estos procesos.
- Las funciones pueden recibir **parámetros**.
- Y también retornan **siempre** un valor. Esto puede hacerse en forma implícita o explícita usando la sentencia **return**.

Ya usamos funciones

- float(), int(), str()
- len(), ord()
- input(), print()

En estos ejemplos, sólo **invocamos** a las funciones ya definidas.

Podemos definir nuestras propias funciones

```
def my_function(param1, param2):
        sentences
        return <expression>
```

- **IMPORTANTE:** el cuerpo de la función debe estar **indentado**.

Una posible función para el primer proceso del desafío:

In [ ]:
```
def input_movies():
    """ This function returns a list with the duration in minutes of the movies """

    minutes = int(input("Ingresá la duración de una película  (0 para finalizar): "))
    movies_duration  = []
    while minutes != 0:
        movies_duration .append(minutes)
        minutes = int(input("Ingresá la duración de una película  (0 para finalizar):

    return movies_duration
```

In [ ]:
```
movies = input_movies()
movies
```

- **¡IMPORTANTE!**
  - Definición vs. invocación.
  - El **docstring**.

El docstring

- Es una secuencia de caracteres que describe la función.
- Se sugiere siempre utilizar triples ".
- **Son procesados por el intérprete.**

In [ ]:
```
help(print)
#help(input_movies)
```

¿Qué pasa si no incluyo el **return**?

In [ ]:
```
def function_without_return():
    var = 10
    print(var)

print(function_without_return())
```

Ahora observemos este código que implementa una posible solución al segundo proceso

In [ ]:
```
def average_calculation(movies_duration):
    """ This function calculates the average of the lengths of the movies received by
    movies_duration: is a list with the duration in minutes of the movies
    """

    len_movies = len(movies_duration)
    average = 0 if len_movies == 0 else sum(movies_duration) / len_movies
    return average
```

In [ ]:
```
average_calculation(movies)
```

- A diferencia de la función anterior, ésta **tiene un parámetro**.
- ¿Hay distintas formas de pasar parámetros en Python? ¿Cómo podemos probar esto?

CONSIGNA: tarea para el hogar

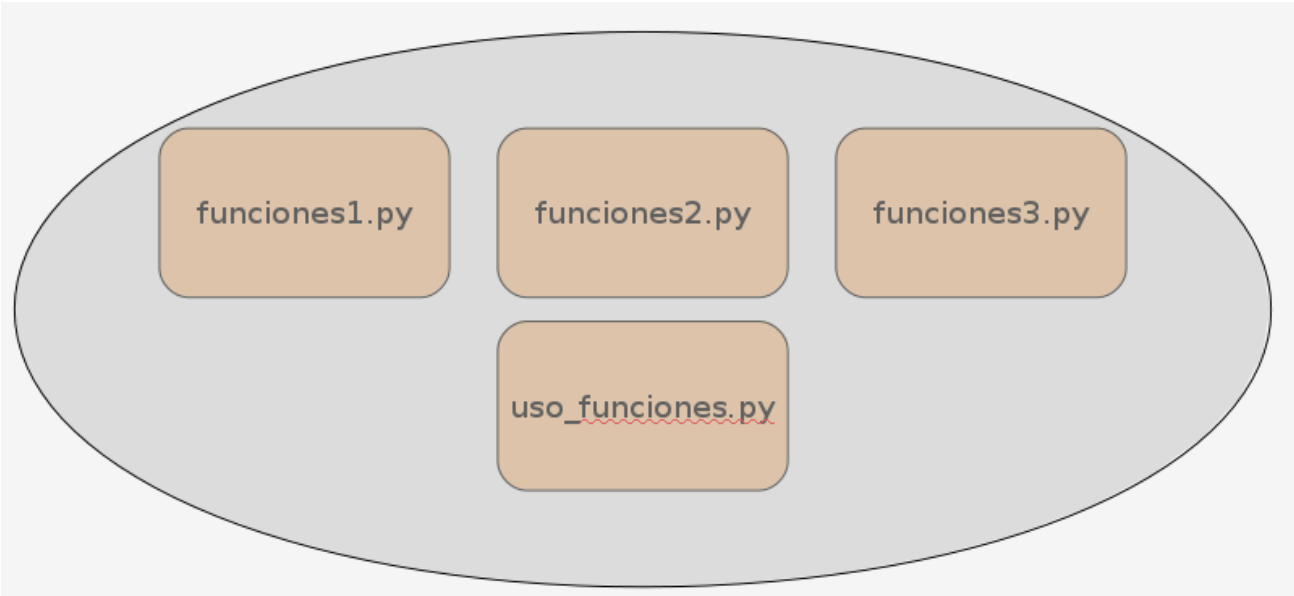
- Definir la función restante para completar el desafío de las películas.

- Escribir un programa que permita probar de qué forma se pasan los parámetros a las funciones en Python.

  PISTA: probar con una función que reciba un entero y otra que reciba una lista.

¿Qué son los módulos en Python?

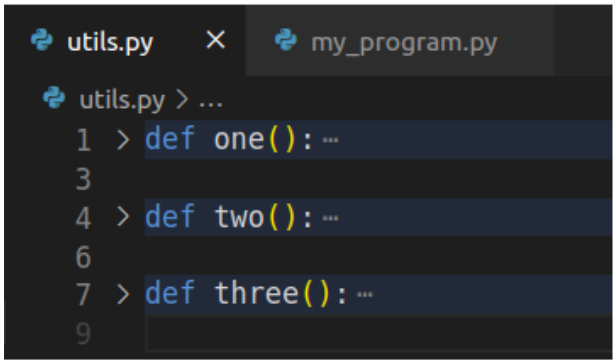
Un módulo es un archivo (con extensión .py) que contiene sentencias y definiciones

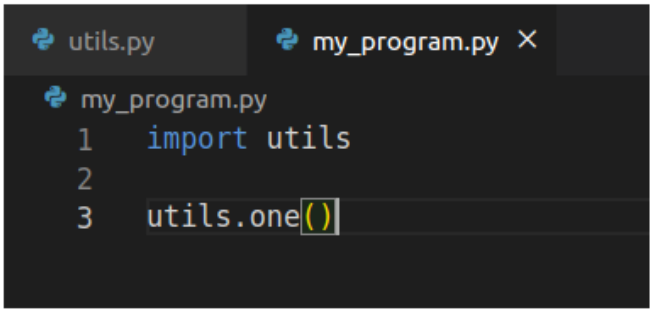


¿Algo nuevo?

Analicemos un ejemplo en el IDE

- ¿Cuántos archivos forman mi programa?
- ¿Cómo se relacionan?





La sentencia import

- Permite acceder a funciones y variables definidas en otro módulo.

In [ ]:
```
import random
```

- Si el módulo contiene definiciones de funciones, sólo se importa estas definiciones, **no las ejecuta**.
- Para ejecutar una función definida en otro módulo **debo invocarla en forma explícita**.

In [ ]:
```
random.randrange(10)
```

Otra forma de importar

```
from my_module import my_function
```

- Sólo se importa **my_function** de **my_module** (no el nombre del módulo).

```
from my_module import *
```

- En este caso, importamos **todos los ítems** definidos en **my_module**.

- **Esta forma no está recomendada**: podrían existir conflictos de nombres. ¿Por qué?

In [ ]:
```
from random import choice
numbers = [x ** 2 for x in range(1, 15, 2)]
choice(numbers)
```

In [ ]:
```
choice = 10
choice
```

Importando módulos

```
import utils

    utils.one()
```

- La importación se realiza **sólo una vez por sesión del intérprete**.
- Veamos sobre un ejemplo.

- Si necesito volver a importar podemos usar **reload()** incluido en el **módulo importlib**.

```
import importlib
    importlib.reload(utils)
```

- +Info en la documentación oficial

Veamos qué pasa cuando queremos importar un módulo que no existe:

In [ ]:
```
import pp
```

¿Dónde se busca los módulos?

- Directorio actual + otros directorios definidos en la variable de ambiente PYTHONPATH

In [ ]:
```
import sys
sys.path
```

Los nombres de los módulos

- Es posible acceder al nombre de un módulo a través de **__name__**

In [ ]:
```
print(__name__)
```

¿Cuándo un módulo se denomina __main__?

- Las instrucciones ejecutadas en el nivel de llamadas superior del intérprete, ya sea desde un script o interactivamente, se consideran parte del módulo llamado **__main__**.

```
#módulo utils
print(f"El nombre de este módulo es {__name__}")

if __name__ == "__main__":
    one()
```

¿Para qué creen que puede ser útil esto?

Agreguemos un menú al ejemplo anterior:

utils.py × my_program.py

utils.py > ...

```
1   def one():
2       print("one")
3
4   def two():
5       print("two")
6
7   def three():
8       print("three")
9
10  print(""" Elegí una opción:
11      (1) one()
12      (2( two()
13      (3) three()
14      """)
15
16  option = int(input())
17  match option:
18      case 1:
19          one()
20      case 2:
21          two()
22      case 3:
23          three()
24
```

utils.py my_program.py ×

my_program.py

```
1   import utils
2
3   utils.one()
```

Primero: ¿qué hace?

Segundo: ¿en qué casos se muestra el menú?

¿Es posible utilizar las funciones sin invocar el menú?

In [ ]:
```
if __name__ == "__main__":
    print(""" Elegí una opción:
    (1) one()
    (2( two()
    (3) three()
    """)
    option = int(input())
    match option:
        case 1:
```

```
            one()
        case 2:
            two()
        case 3:
            three()
```

Ahora: ¿en qué casos se muestra el menú?

CONSIGNA: tarea para el hogar

Reimplementar el desafío de las películas usando funciones pensando que estas funciones se invocan desde el módulo donde están definidas o pueden invocarse desde desde otros módulos.

- Los que quieran, pueden subir el código a su repositorio en GitHub y compartir el enlace a las cuentas **@clauBanchoff** y **@sofiamartin**.

Seguimos la próxima ...